

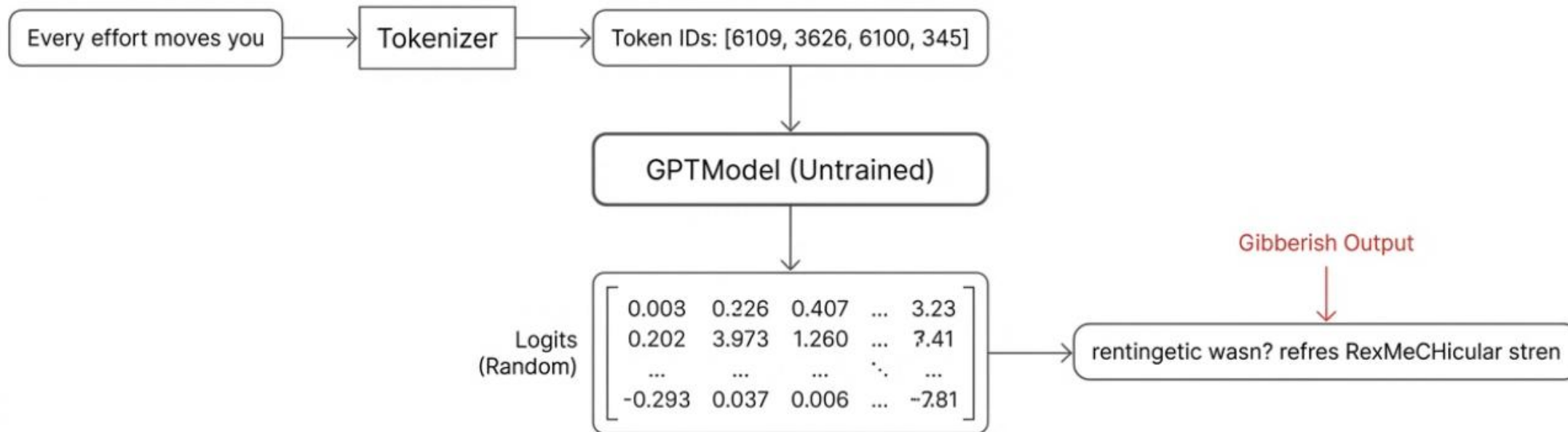


第5章： 在无标签数据上进行预训练

未训练的模型：随机的“混沌大脑”

在训练开始之前，模型的权重通常是**随机初始化**的。此时它具备“计算能力”，能把输入送进网络做一连串矩阵运算，但几乎没有任何语言知识或规律可用。即使输入是一句连贯的话，例如“Every effort moves you”，信号在层与层之间也只是穿过一堆随机数字的变换，最终输出往往是毫无意义的乱码。

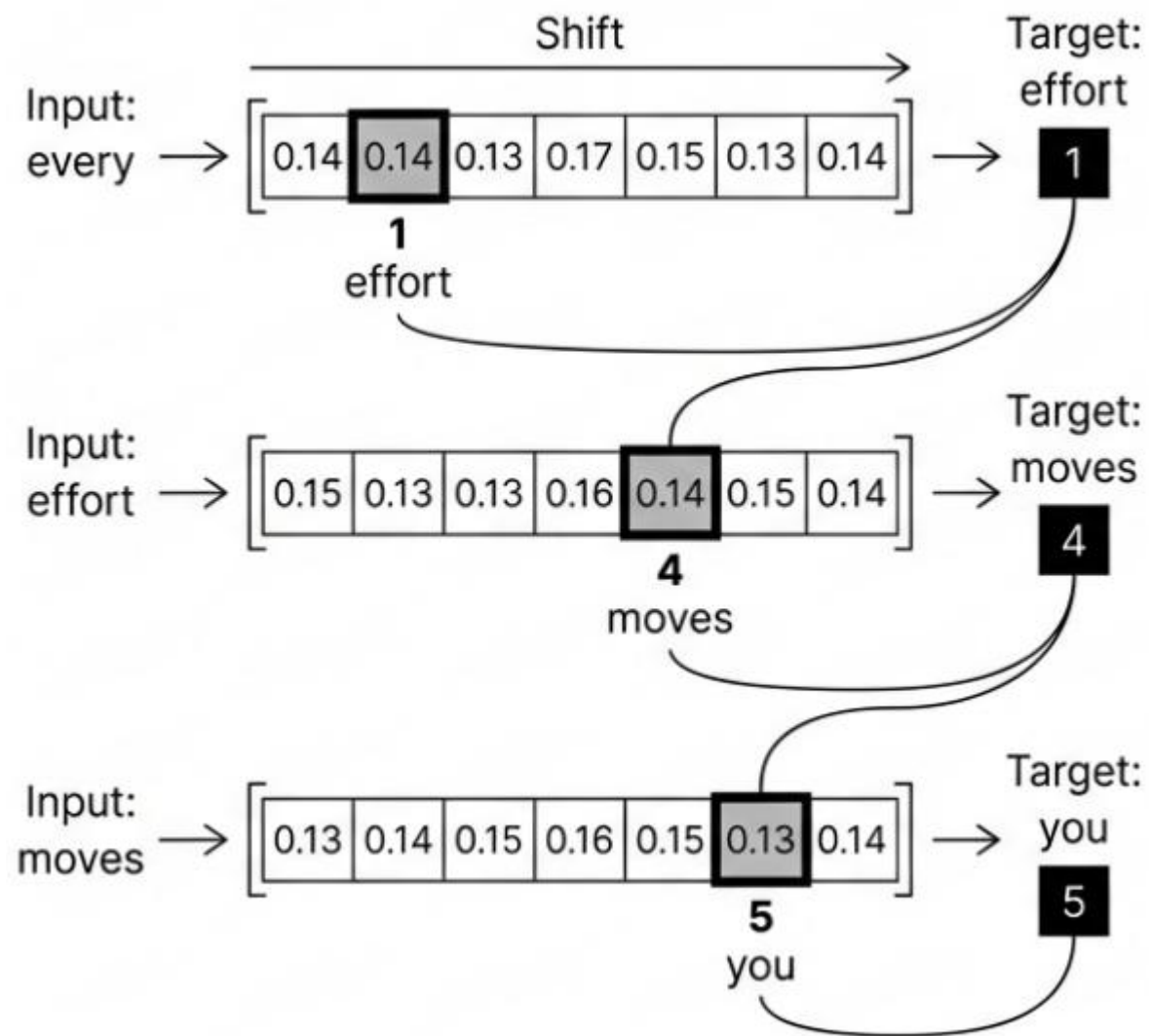
这时模型像是在“乱猜音节”或随意拼接词片段，因为它尚未从数据中学会词与词之间的统计关系、语法结构和上下文依赖。



学习目标：预测下一个 Token

为了训练模型，我们会把原始文本改造成一种“输入—目标错位”的监督形式：对输入序列中的每一个 token 来说，它的**正确答案 (target)** 就是紧跟在后面的那个 token。也就是说，模型看到前面的内容，要学会预测下一步会出现什么。

输入 (x) 是 “every effort moves”，对应的目标 (y) 就是把序列整体右移一位后的 “effort moves you”。训练时，模型会为词表中所有词输出分数，并通过 Softmax 转成概率分布；但计算损失时，我们只关心**模型给“目标 token 的 ID”分配了多大的概率**，目标越接近 1，说明预测越正确。其他错误词的分数不会被当作正确答案，只会通过损失函数间接地被压低。



给模型打分：交叉熵损失

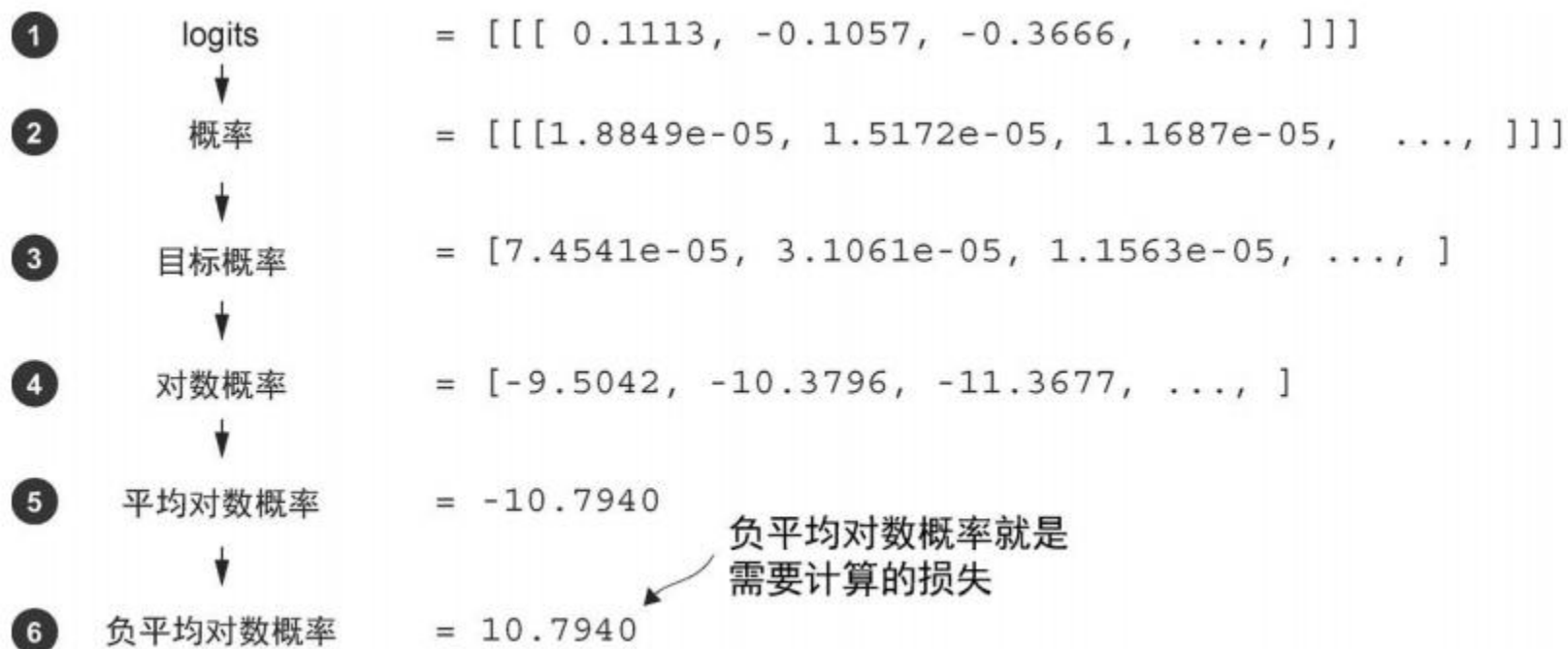
核心原理

模型会对正确答案（正确的下一个词）给出一个预测概率，然后我们对这个概率取负对数（negative log）。

· 如果模型对正确答案的概率是 100%，也就是非常确定，那么损失为 0；

· 如果模型给正确答案的概率很低、很不确定，那么损失就会变得很大。

训练时，优化器的目标就是**不断调整参数，让这个失败分数逐步下降**，从而让模型越来越倾向于把高概率分配给正确的词。



学习的循环：训练流程 Training Loop

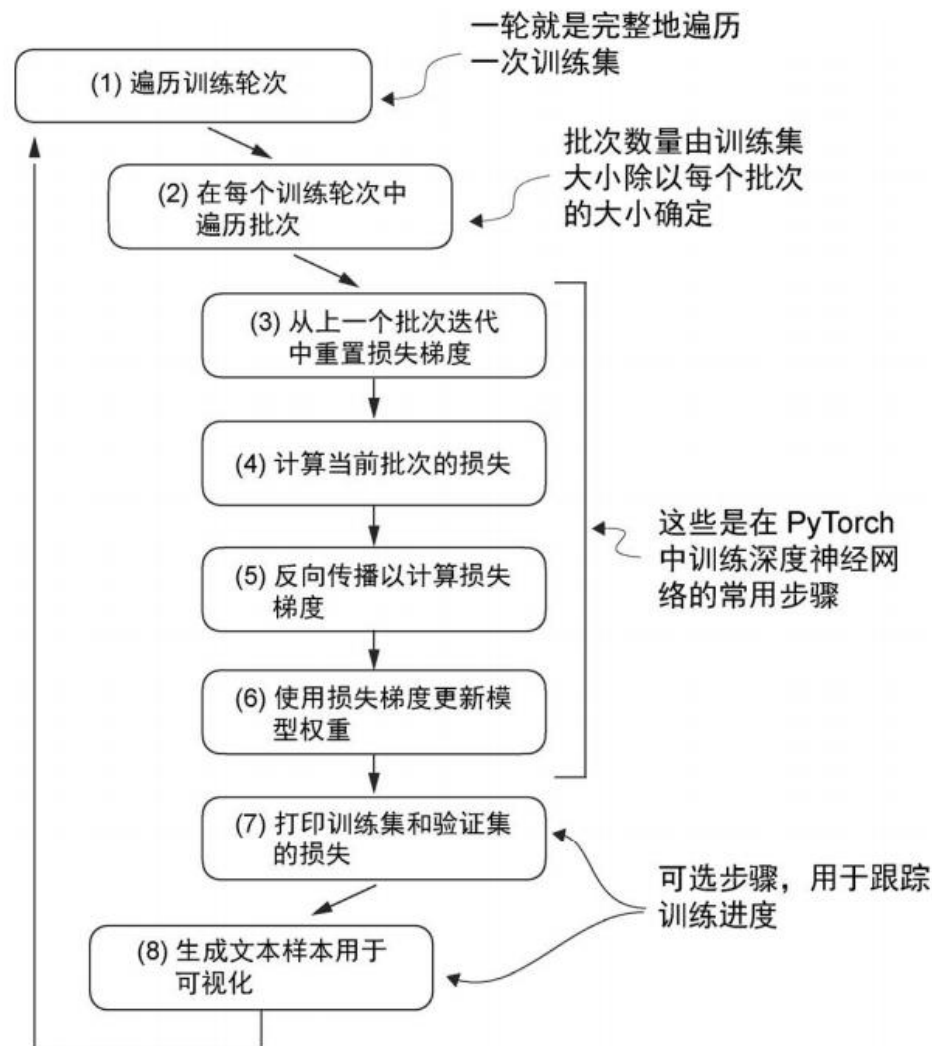
预训练本质上是一个不断“犯错—纠错—改进”的迭代循环。

(1) 前向传播 (Forward Pass)：模型在一批文本 (batch) 上根据上下文猜测下一个 token。

(2) 计算损失 (Calculate Loss)：把模型的预测与真实目标进行对比，得到误差大小。

(3) 反向传播 (Backward Pass)：计算梯度，把误差追溯到网络中哪些参数和神经元导致了这些错误。

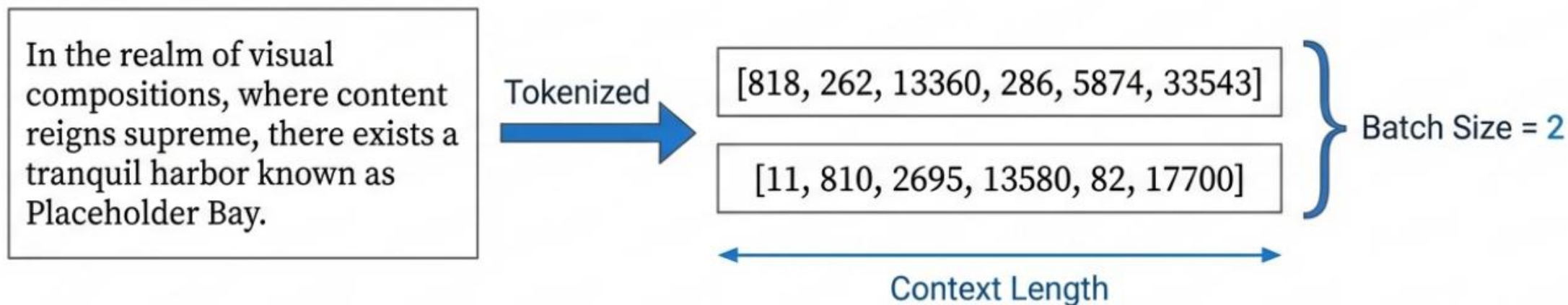
(4) 更新权重 (Update Weights)：优化器根据梯度对参数做小幅调整，让模型在下一次遇到类似输入时更不容易出错。



Batch 与上下文窗口 Context

我们不可能一次把整个训练语料都喂进模型。实际训练时，会先用“滑动窗口”的方式把长文本切成可管理的片段，每个片段的长度由**上下文长度 (context length)** 决定。这样做既能控制计算量，也能让模型在固定窗口内学习词与词之间的关系。

切好的这些片段会再被堆叠成**批次 (batches)**。一个 batch 里包含多条序列，GPU 可以并行处理它们，从而显著提高吞吐量，加速训练过程。



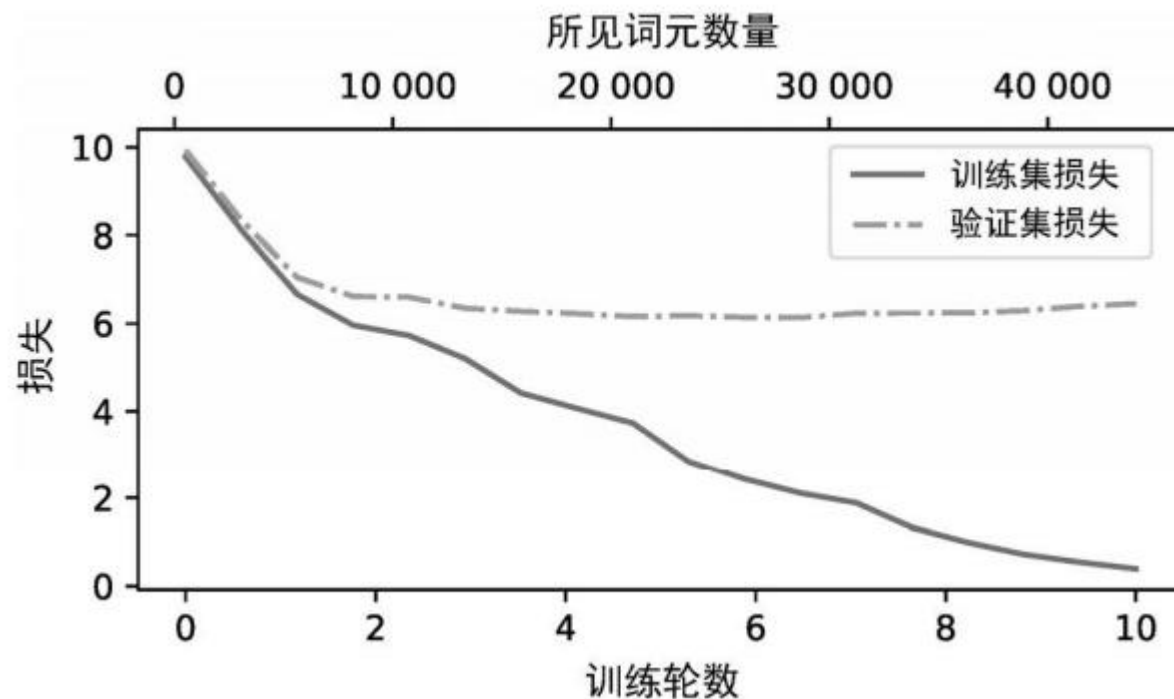
记忆 vs 理解：看懂损失曲线 Loss Curves

为了判断模型是在真正学习规律，还是只是在“背题”，训练时通常会同时跟踪两个指标：训练损失和验证损失。

训练损失（实线）衡量模型在已经见过的训练文本上的表现，它一般会随着训练不断下降，这是正常现象。

验证损失（虚线）衡量模型在未见过的新文本上的表现，更能代表真实的泛化能力。

需要重点观察的是两条曲线是否出现“分叉”。如果训练损失还在持续下降，但验证损失不再下降甚至开始上升，这通常意味着模型发生了**过拟合（Overfitting）**：它在机械记忆训练数据的细节，而不是学到可迁移的语言规律，因此在新数据上反而表现变差。



核心原理

面对一堆“下一个词的概率”，到底该选哪一个词输出。不同的解码策略会直接影响文本的风格与质量。

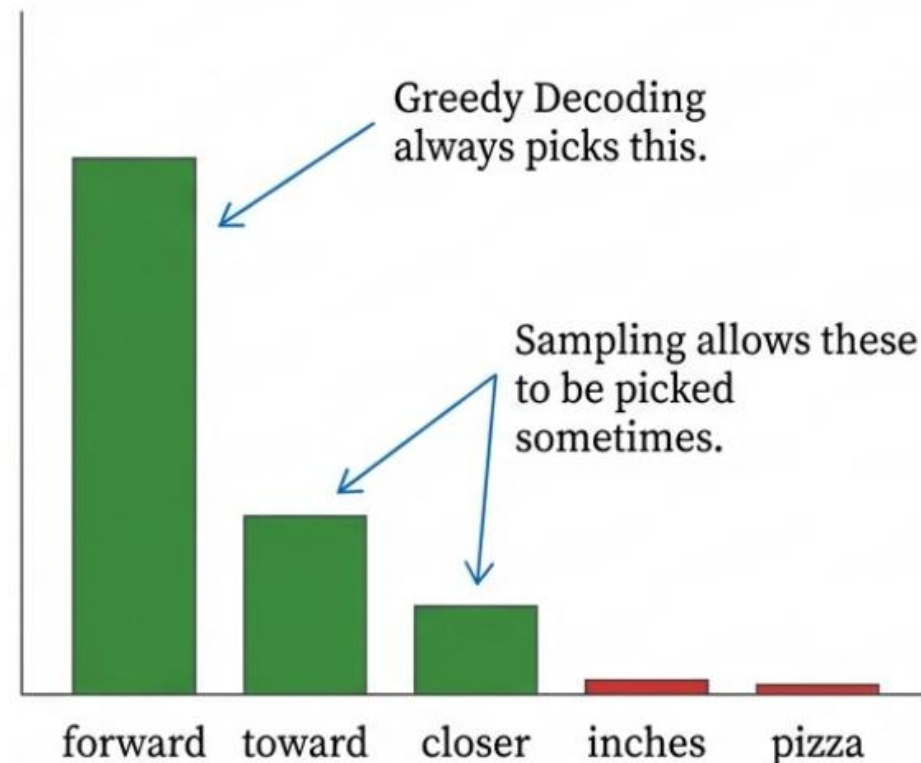
贪心解码 (Greedy Decoding, 温度 Temp=0)

每一步都只选择概率最高的那个词。这种方式很稳、很少出错，但往往显得机械，容易产生重复和循环，读起来不够自然。

概率采样 (Probabilistic Sampling)

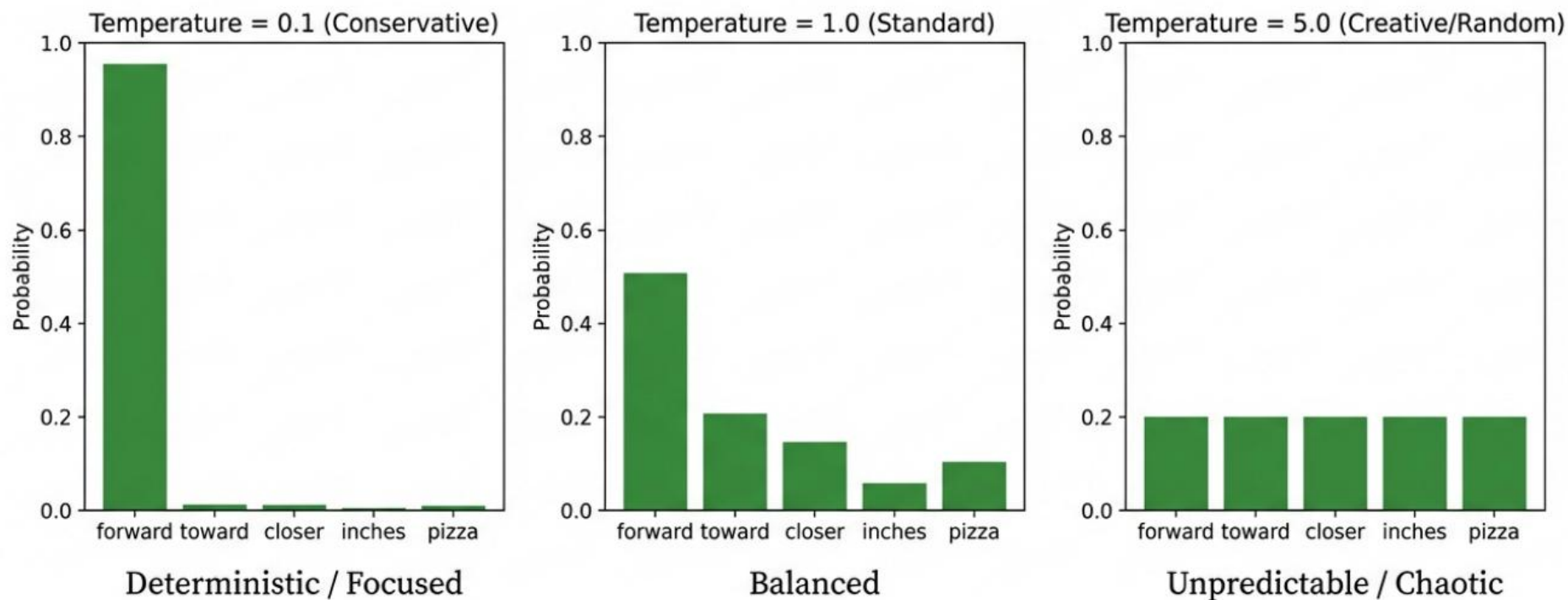
则像掷骰子一样按概率抽取词：高概率的词更容易被选中，但“第二好、第三好”的选项也偶尔会出现。这样能带来更接近人类的多样性与创造性，让生成内容更灵活、更有变化。

Next Word Probabilities after:
“Every effort moves you...”

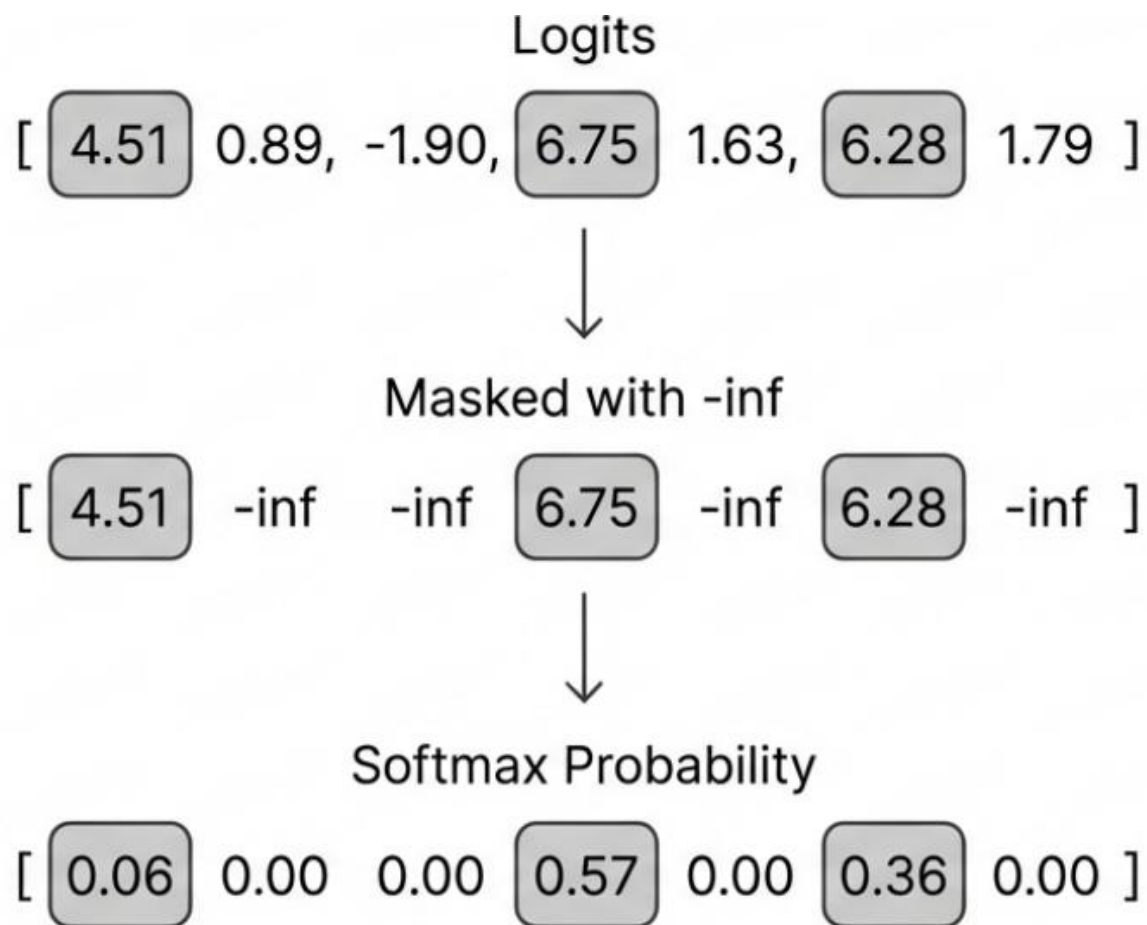


温度缩放 Temperature Scaling

温度 (Temperature) 用来调节模型输出概率分布的“尖锐程度” (sharpness)。温度越低，分布越尖锐，模型更倾向于选择最高概率的词，输出更稳定但更保守；温度越高，分布越平坦，低概率词也更有机会被采样到，文本会有更多变化、更具创造性，但也更容易跑题或产生不连贯的内容。



Top-k 采样 Top-k Sampling



Top-k 采样是一种控制生成风险的解码策略，可以把它理解为给模型加上“安全护栏”。在每一步生成时，**模型先只保留概率最高的前 k 个候选词，把其余低概率词全部屏蔽掉**；然后只在这 k 个候选里按概率进行采样。

这样既能保留一定的随机性与多样性，又能避免模型从大量极低概率的词里“乱选”，从而减少离谱输出、提升整体连贯性与可控性。

保存模型能力：State Dictionary (state_dict)



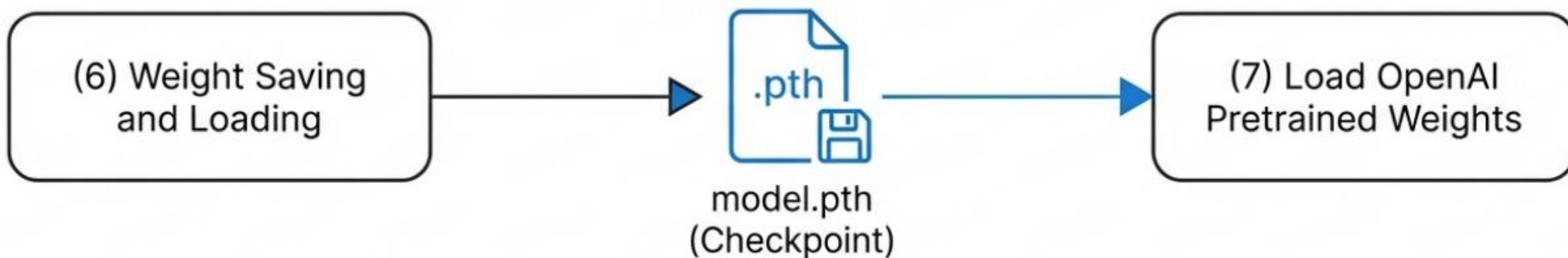
LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

训练大模型的计算成本很高，因此必须能随时保存进度，避免中断后从头再来。在 PyTorch 里，**state_dict** 是一个 Python 字典，用来把模型的每一层映射到它学到的张量参数。你可以把它理解为模型的一张“记忆快照”，把当前学到的知识完整记录下来。

通过保存 state_dict，我们可以把训练过程中的检查点 (checkpoint) 写入磁盘，常见文件扩展名是 .pth。之后无论是继续训练 (resume training)，还是把模型加载出来用于部署与推理，都可以直接从这些检查点恢复参数，从而保留已有的训练成果。



使用预训练权重

从零开始训练一个大语言模型需要极其庞大的数据与算力，通常只有大型数据中心级别的资源才能承担。更现实的做法是复用已经训练好的预训练权重，例如 OpenAI 发布的 GPT-2 权重。只要我们的模型架构与 GPT-2 对齐，就可以把这些权重直接加载进来。

模型会从一个“随机猜测器”瞬间升级为能够流畅生成英文文本的系统。也就是说，我们不需要重新学会基础语言规律，而是直接继承了预训练阶段已经学到的大量统计模式与语言结构，然后再根据自己的任务做进一步微调或应用开发。

- GPT-2 Small: 124M Params (12 Layers)
- GPT-2 Medium: 355M Params (24 Layers)
- GPT-2 Large: 774M Params (36 Layers)
- GPT-2 XL: 1.5B Params (48 Layers)

N x

GPT Model Architecture

Output Layer

Transformer Blocks

Input Embedding